
26 Parallel Algorithms

The vast majority of algorithms in this book are *serial algorithms* suitable for running on a uniprocessor computer that executes only one instruction at a time. This chapter extends our algorithmic model to encompass *parallel algorithms*, where multiple instructions can execute simultaneously. Specifically, we'll explore the elegant model of task-parallel algorithms, which are amenable to algorithmic design and analysis. Our study focuses on fork-join parallel algorithms, the most basic and best understood kind of task-parallel algorithm. Fork-join parallel algorithms can be expressed cleanly using simple linguistic extensions to ordinary serial code. Moreover, they can be implemented efficiently in practice.

Parallel computers—computers with multiple processing units—are ubiquitous. Handheld, laptop, desktop, and cloud machines are all *multicore computers*, or simply, *multicores*, containing multiple processing “cores.” Each processing core is a full-fledged processor that can directly access any location in a common *shared memory*. Multicores can be aggregated into larger systems, such as clusters, by using a network to interconnect them. These multicore clusters usually have a *distributed memory*, where one multicore's memory cannot be accessed directly by a processor in another multicore. Instead, the processor must explicitly send a message over the cluster network to a processor in the remote multicore to request any data it requires. The most powerful clusters are supercomputers, comprising many thousands of multicores. But since shared-memory programming tends

to be conceptually easier than distributed-memory programming, and multicore machines are widely available, this chapter focuses on parallel algorithms for multicores.

One approach to programming multicores is *thread parallelism*. This processor-centric parallel-programming model employs a software abstraction of “virtual processors,” or *threads* that share a common memory. Each thread maintains its own program counter and can execute code independently of the other threads. The operating system loads a thread onto a processing core for execution and switches it out when another thread needs to run.

Unfortunately, programming a shared-memory parallel computer using threads tends to be difficult and error-prone. One reason is that it can be complicated to dynamically partition the work among the threads so that each thread receives approximately the same load. For any but the simplest of applications, the programmer must use complex communication protocols to implement a scheduler that load-balances the work.

Task-parallel programming

The difficulty of thread programming has led to the creation of *task-parallel platforms*, which provide a layer of software on top of threads to coordinate, schedule, and manage the processors of a multicore. Some task-parallel platforms are built as runtime libraries, but others provide full-fledged parallel languages with compiler and runtime support.

Task-parallel programming allows parallelism to be specified in a “processor-oblivious” fashion, where the programmer identifies what computational tasks may run in parallel but does not indicate which thread or processor performs the task. Thus, the programmer is freed from worrying about communication protocols, load balancing, and other vagaries of thread programming. The task-parallel platform contains a scheduler, which automatically load-balances the tasks across the processors, thereby greatly simplifying the programmer’s chore. *Task-parallel algorithms* provide a natural extension to ordinary

serial algorithms, allowing performance to be reasoned about mathematically using “work/span analysis.”

Fork-join parallelism

Although the functionality of task-parallel environments is still evolving and increasing, almost all support *fork-join parallelism*, which is typically embodied in two linguistic features: *spawning* and *parallel loops*. Spawning allows a subroutine to be “forked”: executed like a subroutine call, except that the caller can continue to execute while the spawned subroutine computes its result. A parallel loop is like an ordinary **for** loop, except that multiple iterations of the loop can execute at the same time.

Fork-join parallel algorithms employ spawning and parallel loops to describe parallelism. A key aspect of this parallel model, inherited from the task-parallel model but different from the thread model, is that the programmer does not specify which tasks in a computation *must* run in parallel, only which tasks *may* run in parallel. The underlying runtime system uses threads to load-balance the tasks across the processors. This chapter investigates parallel algorithms described in the fork-join model, as well as how the underlying runtime system can schedule task-parallel computations (which include fork-join computations) efficiently.

Fork-join parallelism offers several important advantages:

- The fork-join programming model is a simple extension of the familiar serial programming model used in most of this book. To describe a fork-join parallel algorithm, the pseudocode in this book needs just three added keywords: **parallel**, **spawn**, and **sync**. Deleting these parallel keywords from the parallel pseudocode results in ordinary serial pseudocode for the same problem, which we call the “serial projection” of the parallel algorithm.
- The underlying task-parallel model provides a theoretically clean way to quantify parallelism based on the notions of “work” and “span.”
- Spawning allows many divide-and-conquer algorithms to be parallelized naturally. Moreover, just as serial divide-and-conquer